

Fast Fourier Transform (FFT) Implementation and Its Applications

MATH307 Numerical Analysis — Final Project

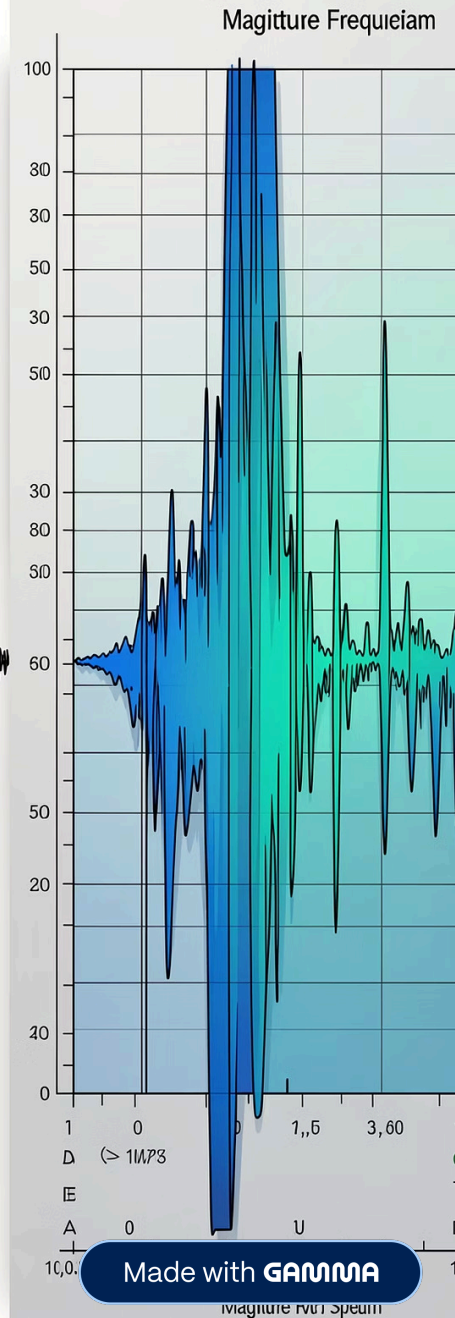
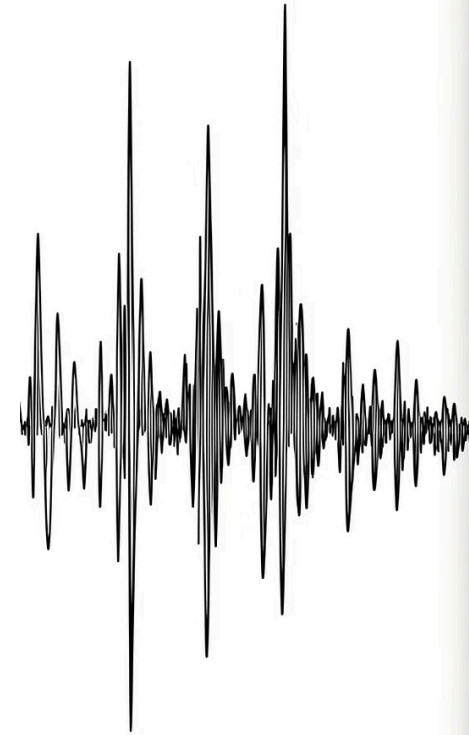
Team: Ali Ashraf (202301812), Abdelrahman Mohamed (202300056),
Ahmed Mohamady (202301826)

Goal

Efficiently compute discrete Fourier transforms and apply them to signal and image processing tasks.

Deliverables

Mathematical derivation, Python implementation, experiments, reproducible figures, final report, demo-ready code.



Motivation & Course Alignment



- Direct DFT is exact but costs $O(N^2)$ — impractical for large N .
- FFT reduces cost to $O(N \log N)$, enabling large-scale spectral analysis.
- Applications: denoising, spectral analysis, image filtering, compression, AI preprocessing.
- Rubric fit: theory, algorithm design, implementation, empirical comparison, parameter study, clear presentation.

Mathematical Foundation: DFT & Inverse DFT

Discrete Fourier Transform

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}$$

Inverse Discrete Fourier Transform

$$x_n = \frac{1}{N} \sum_{k=0}^{N-1} X_k e^{i2\pi kn/N}$$

Matrix Form

$$X = W_N x$$

Direct evaluation is a dense matrix-vector multiplication, so it costs $O(N^2)$.

FFT Methodology & Complexity

01

Divide

Split sequence into even and odd-indexed samples.

02

Conquer (butterfly)

FFT Butterfly Relations

$$X_k = E_k + \omega_N^k O_k$$

$$X_{k+N/2} = E_k - \omega_N^k O_k$$

where $\omega_N = e^{-i2\pi/N}$ are twiddle factors

03

Recurrence & Cost

Runtime Recurrence

$$T(N) = 2T(N/2) + O(N)$$

Therefore: $T(N) = O(N \log N)$

FFT reorganizes exact DFT arithmetic; not an approximation.

Implementation & Validation Workflow

Implemented:

- Direct DFT from definition (reference implementation)
- Recursive radix-2 Cooley–Tukey FFT and inverse via conjugation
- Validation against NumPy FFT using relative error metrics
- Automated figure generation for the report (Report/generated/)

```
def fft(x):  
    N = len(x)  
    if N==1: return x  
    even = fft(x[0::2])  
    odd = fft(x[1::2])  
    return [even[k] + tw[k]*odd[k] for k in range(N//2)] + \  
           [even[k] - tw[k]*odd[k] for k in range(N//2)]
```

Benchmark Results — Runtime & Numerical Accuracy

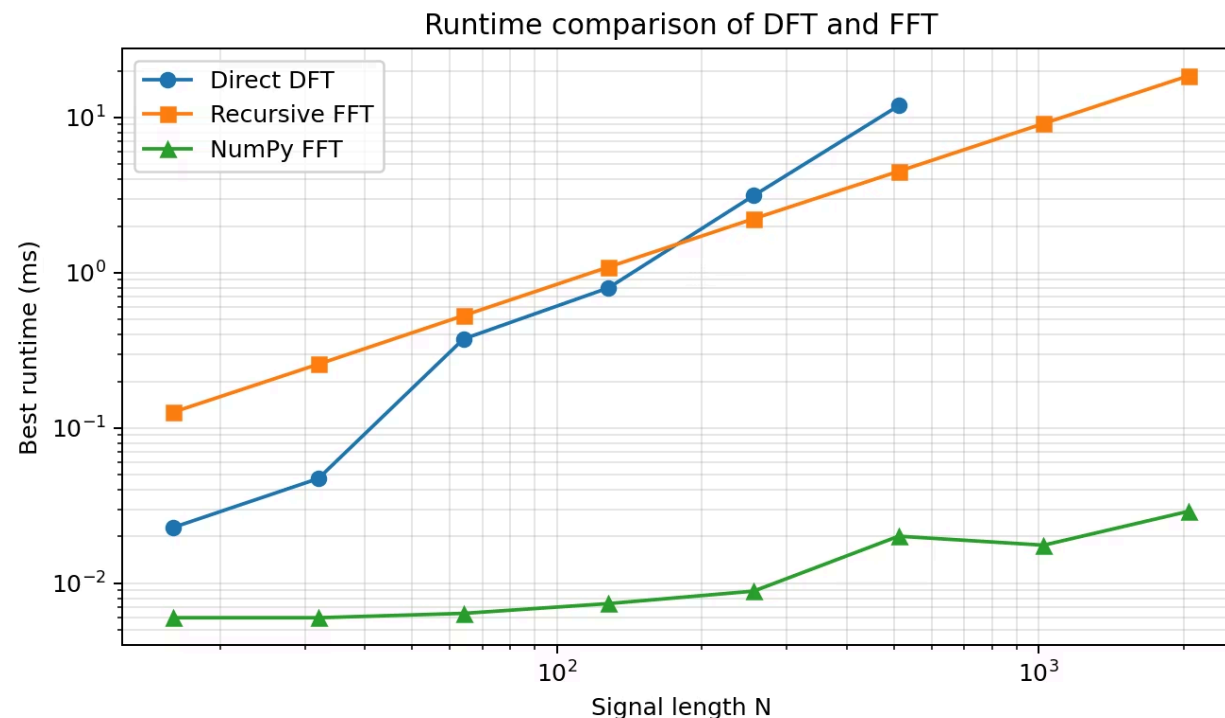
Tested complex random signals N=16..2048. Representative (N=512):

Representative timings

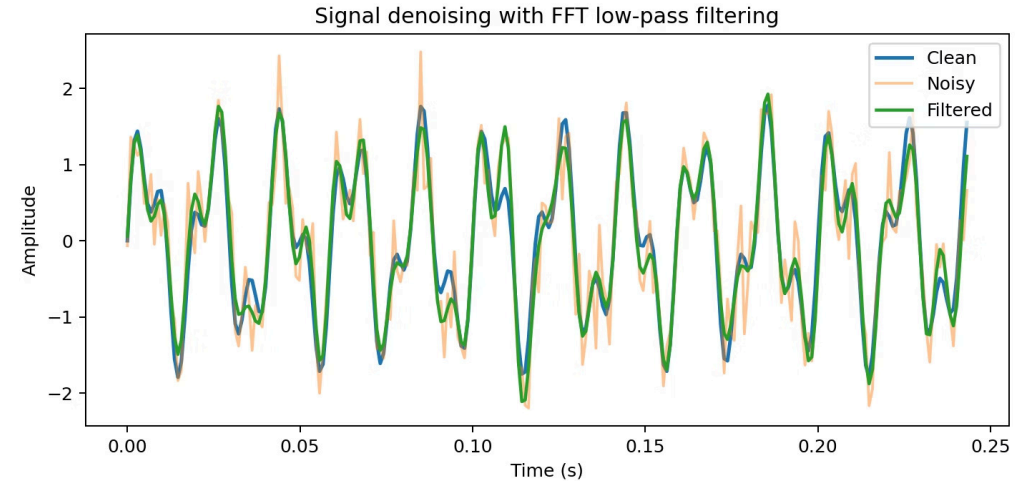
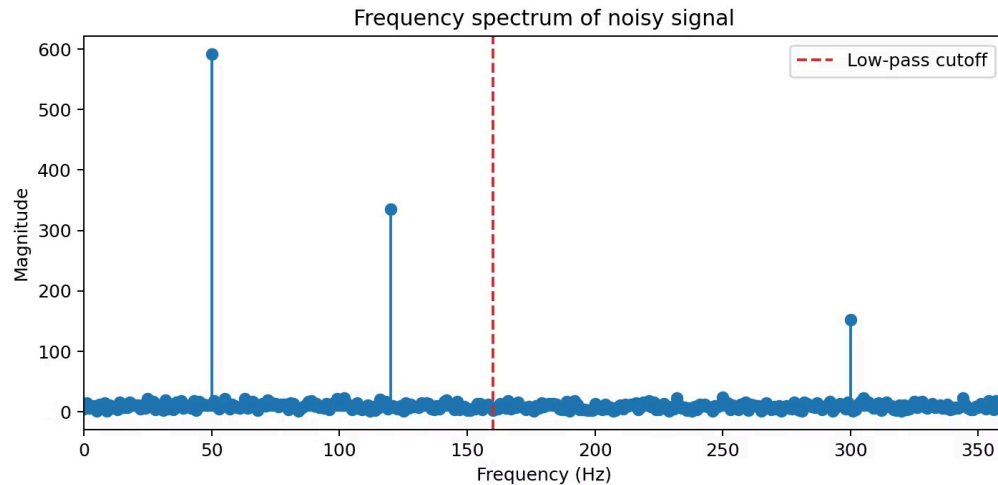
Direct DFT: 11.915 ms — Recursive FFT: 4.503 ms —
NumPy FFT: 0.0201 ms

Numerical accuracy

Recursive FFT relative error $\approx 10^{-15}$ (machine precision).
NumPy FFT fastest due to optimized compiled routines.



Signal Processing Experiment — Spectrum & Denoising



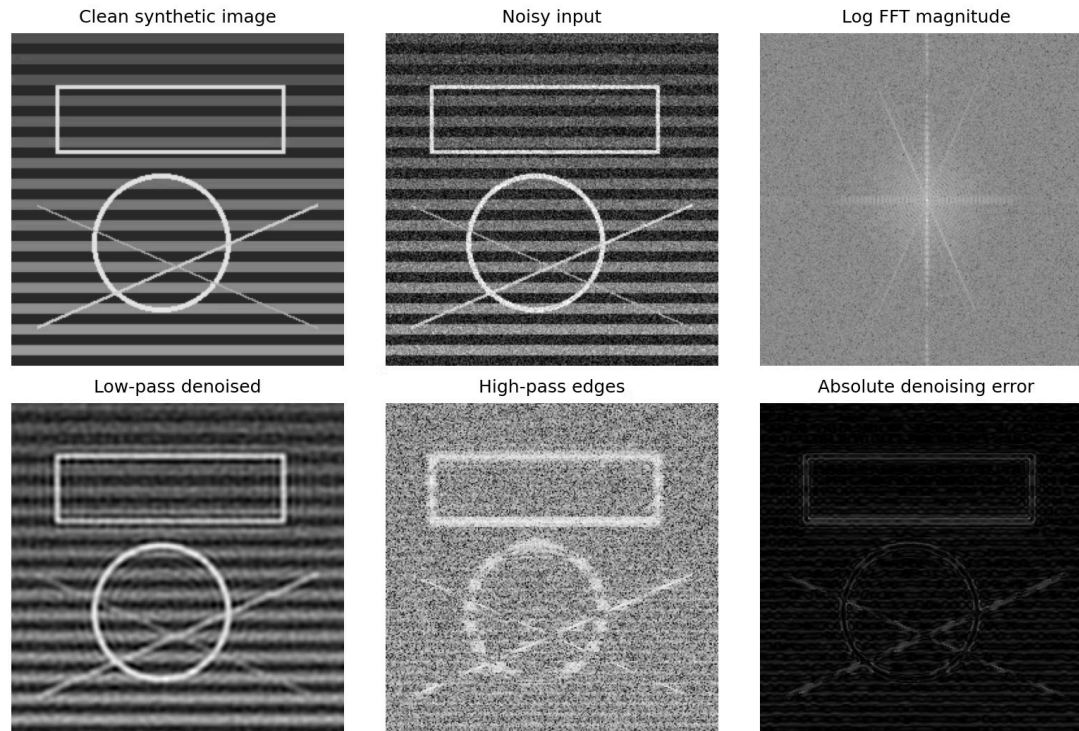
Setup: clean tones at 50 Hz & 120 Hz, added 300 Hz interference + Gaussian noise.

Results: Dominant frequencies detected — 50 Hz, 120 Hz, 300 Hz. Low-pass cutoff 160 Hz removed 300 Hz interference.

MSE & SNR

Signal noisy MSE: 0.1579 → filtered MSE: 0.0375. Improvement: 6.24 dB.

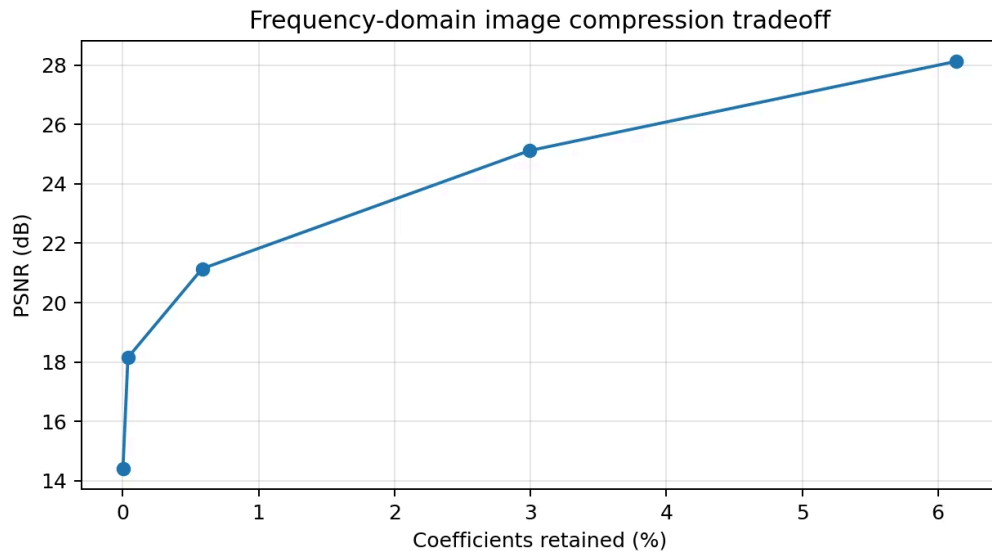
Image Processing Experiment — 2D FFT Filtering



2D FFT separates image into frequency bands:
low frequencies = smooth structure; high
frequencies = edges and noise.

- Image noisy PSNR: 20.65 dB → low-pass filtered PSNR: 21.99 dB
- Image noisy MSE: 560.29 → filtered MSE: 411.54
- Low-pass mask retained: 8.43% of centered frequency coefficients

Compression, Parameter Effects & Error Analysis



Compression tradeoff

98% energy $\rightarrow$ 2.99% coefficients $\rightarrow$ 25.12 dB PSNR. 99% energy $\rightarrow$ 6.13% coefficients $\rightarrow$ 28.13 dB PSNR.



Error sources

Numerical error: $\approx$ machine precision.
Modeling/filtering error: dominates perceptual loss (cutoffs, mask radius).

Practical guidance: choose cutoff/mask to balance denoising vs. detail preservation; verify stability across N and windowing choices.

Demo Readiness, Conclusions & References

Reproducibility

Run: python
experiments/fft_experiments.py — outputs written to
Report/generated/ (plots and
metrics included).

Key conclusions

- FFT reduces DFT complexity from $O(N^2)$ to $O(N \log N)$ while preserving accuracy.
- Empirical results: significant runtime gains and machine-precision-level numerical error for radix-2 recursive FFT.
- Applications confirmed: signal denoising, image filtering, edge emphasis, and compression.

References

- Cooley, J. W., & Tukey, J. W. (1965). An algorithm for the machine calculation of complex Fourier series.
- Oppenheim, A. V., & Schafer, R. W., Discrete-Time Signal Processing.
- Smith, S. W., The Scientist and Engineer's Guide to DSP.
- Gonzalez, R. C., & Woods, R. E., Digital Image Processing.
- NumPy FFT documentation.

Final demo assets

Report/generated/ contains runtime_comparison.png,
signal_spectrum.png, signal_filtering.png,
image_fft_processing.png, compression_tradeoff.png